

International Institute of Information Technology
Hyderabad, India

Improving SDN Forensic Data Storage using Cuckoo Filters

PG Project Report

Submitted by

Kuldeep Kumar Lakhe Roll No: 2024202025

Atharva Keskar Roll No: 2024202017

Dheeraj Makhija Roll No: 2024202011

Under the Guidance of

Prof. Shatrunjay Rawat

International Institute of Information Technology, Hyderabad

shatrunjay.rawat@iiit.ac.in

April 2026

Abstract

Network forensic analysis in Software Defined Networking (SDN) environments depends critically on the continuous availability of forensic log data. Existing approaches, notably the work by Sharma and Rawat, use Bloom Filters to store and compress SDN forensic logs efficiently. However, Bloom Filters suffer from a fundamental limitation: they do not support selective deletion of individual entries. When a Bloom Filter reaches capacity, the entire filter must be wiped and reset, resulting in complete and irreversible loss of all stored forensic evidence. Furthermore, even Cuckoo Filter-based approaches that use sliding window deletion are forensically unsound, as modifying forensic logs violates evidence immutability requirements.

This project proposes a three-tier improvement over existing approaches. First, we replace Bloom Filters with Cuckoo Filters and implement an *Attack-Aware Sliding Window* retention policy. Second, we introduce a **Forensic Filter Manager** that uses a *Snapshot + Reset* archival model: when a filter approaches capacity, it is serialized to disk as an immutable snapshot and a fresh filter is created — ensuring that forensic evidence is **never deleted or modified**. Third, we extend the SDN topology from a linear chain to a realistic **hierarchical tree topology** (core–aggregation–access) and simulate **multiple simultaneous attacks** from different sources. We implement two complementary traceback mechanisms: (i) a Source Path Isolation-style **query-all** traceback, and (ii) a **parent-pointer walk-back** in which each router remembers which upstream router handed it the packet, allowing the victim (through an SDN controller) to reconstruct the exact ordered attack path. The entire pipeline is evaluated on the CIC-IDS2017 benchmark dataset containing real-world FTP-Patator brute-force attacks across 50,000 packet records.

Our experiments produce six concrete findings. First, in the offline filter comparison, the Cuckoo Filter with sliding window achieves **96/100** traceability vs. **2/100** for Bloom. Second, the **Snapshot + Reset** model achieves **100/100** traceability with zero deletions and zero wipes — fully forensically sound. Third, on a **7-switch tree topology** with two simultaneous attack types ($H_0 \rightarrow H_3$ and $H_1 \rightarrow H_2$), Cuckoo-based traceback achieves **100/100** success while Bloom achieves only **1/100**. Fourth, a **fingerprint size sweep** (8–16 bits) shows that increasing from 10 to 16 bits reduces the false positive rate from 0.78% to 0.01% at a cost of only +60% memory per filter. Fifth, parent-pointer traceback issues only **O(path length)** queries vs. SPIE’s **O(N switches)**, a **42% reduction** in a 6-switch network and **17×** reduction at 50 switches. Sixth, a scalability sweep from **N=6**

to $N=50$ shows accuracy remains at **99–100%**.

Keywords: SDN, Network Forensics, Cuckoo Filter, Bloom Filter, Forensic Immutability, Snapshot + Reset, Tree Topology, Parent-Pointer Traceback, SPIE, SDN Controller, Attack-Aware Eviction, Fingerprint Sweep, Scalability, CIC-IDS2017

GitHub Repository:  [Repository Link](#)

Acknowledgement

We would like to express our sincere gratitude to **Prof. Shatrunjay Rawat**, our project guide at the International Institute of Information Technology, Hyderabad, for his invaluable guidance, continuous support, and encouragement throughout this project. His foundational research work on network management frameworks and Bloom Filter-based SDN forensic log retention directly inspired and shaped our contribution. The idea of *each soft router remembering which neighbour handed it the packet*, which became the parent-pointer traceback mechanism in this work, was suggested during our project meetings with him.

We are also grateful to the Canadian Institute for Cybersecurity at the University of New Brunswick for making the CIC-IDS2017 dataset publicly available for research purposes.

Kuldeep Kumar Lakhe

Atharva Keskar

Dheeraj Makhija

IIIT Hyderabad, April 2026

Contents

Abstract	1
Acknowledgement	3
List of Figures	8
List of Tables	8
1 Introduction	9
1.1 Background	9
1.2 Problem Statement	9
1.3 Proposed Solution	10
1.4 Project Objectives	10
1.5 Report Organization	11
2 Related Work	12
2.1 Network Management Framework for Forensic Traceability	12
2.2 Source Path Isolation Engine (SPIE)	12
2.3 Bloom Filter-Based SDN Forensic Log Retention	13
2.4 Cuckoo Filters	13
3 System Design	15
3.1 System Overview	15
3.2 Bloom Filter Implementation	16
3.3 Cuckoo Filter Implementation	17
3.4 Attack-Aware Sliding Window Retention Policy	17
3.5 Forensic Filter Manager: Snapshot + Reset Model	18
3.6 Configurable Fingerprint Size	19
3.7 Design Tradeoffs and Parameter Justification	20
3.7.1 Why Fingerprint Size = 10 Bits (Default)	20
3.7.2 Why 12 Bits Is the Recommended Sweet Spot	20
3.7.3 Why Bucket Size = 4	21
3.7.4 Why Sliding Window Threshold = 90%	21

3.7.5	Why Snapshot Trigger = 85%	22
3.7.6	Why 6 Hash Functions for Bloom Filter	22
3.7.7	Why Max Kicks = 500	22
3.7.8	Why Victim Stash Size = 10	23
3.7.9	Why Filter Capacity = 10,000 Items	23
3.7.10	Summary of Design Tradeoffs	23
3.8	Tree Topology (Core–Aggregation–Access)	24
3.9	Flow Key Construction	25
3.10	SDN Soft-Router Network Topology	25
3.11	Traceback Strategy 1: SPIE Query-All	26
3.12	Traceback Strategy 2: Parent-Pointer Walk-Back	26
3.13	SDN Controller Orchestration	27
3.14	Traceability Definitions	28
4	Experimental Methodology	30
4.1	Dataset	30
4.2	Experiment 1: Offline Filter Comparison	30
4.3	Experiment 2: SDN Soft-Router Traceback	31
4.4	Experiment 3: Scalability Sweep	32
4.5	Experiment 4: Enhanced Three-Way Comparison	32
4.6	Experiment 5: Tree Topology with Multiple Attacks	33
4.7	Experiment 6: Fingerprint Size Sweep (Cost-Benefit)	33
4.8	Implementation Environment	33
5	Results and Analysis	34
5.1	Offline Comparison: Performance Graphs	34
5.2	Evidence Traceability (Offline)	34
5.3	Memory Usage (Offline)	35
5.4	Wipes vs. Selective Deletes (Offline)	35
5.5	False Positive Rate (Offline)	35
5.6	Empirical False Positive Rate Test (Offline)	36
5.7	Offline Summary	36
5.8	Network Traceback Results	36
5.9	Network-Level False Positive and Negative Rates	38
5.10	Controller Query Efficiency	38
5.11	Scalability	39
5.12	Enhanced Three-Way Comparison (Snapshot + Reset)	40
5.13	Tree Topology with Multiple Simultaneous Attacks	40
5.14	Fingerprint Size Sweep: Cost-Benefit Analysis	42

6	Discussion	43
6.1	Answering the Traceability Question	43
6.2	The Attack-Aware Eviction Advantage	43
6.3	Why Parent-Pointer Preserves Order (and SPIE Doesn't)	44
6.4	Concrete Use Cases	44
6.5	Limitations	45
7	Conclusion and Future Work	46
7.1	Conclusion	46
7.2	Future Work	47
8	Contributions and Work Distribution	48

List of Figures

3.1	High-level system architecture. The <i>filter layer</i> (per switch) provides the Bloom baseline and our Attack-Aware Cuckoo Filter, including the <code>parent_sw_id</code> metadata. The <i>network layer</i> composes these filters into a soft-router topology coordinated by an SDN controller that queries the switches on behalf of the victim.	16
3.2	SDN soft-router topology used in the network experiment. Six switches $S_0, \dots, S_5$ form a linear backbone with one cross-link between S_1 and S_4 . Hosts H_0 (attacker) and H_3 (victim) are attached at S_0 and S_3 respectively. The red edges mark the attack path $H_0 \rightarrow S_0 \rightarrow S_1 \rightarrow S_2 \rightarrow S_3 \rightarrow H_3$	26
3.3	Parent-pointer walk-back. Starting at the victim's ingress switch S_3 , the controller asks each switch for the stored <code>parent_sw_id</code> and hops to that switch in turn. The walk terminates at S_0 where <code>parent_sw_id = None</code> , identifying the attacker's ingress point. Only the 4 on-path switches are visited — not all N switches.	27
3.4	Controller-orchestrated investigation sequence. On receiving the victim's excerpt, the controller executes all three traceback strategies. Bloom + SPIE and Cuckoo + SPIE broadcast $O(N)$ queries; Cuckoo + Parent-Pointer follows the stored <code>parent_sw_id</code> pointers for $O(\text{path length})$ queries. The controller returns the reconstructed paths and per-strategy query counts.	28
5.1	Comparative performance of Bloom Filter vs. Cuckoo Filter across 50,000 CIC-IDS2017 packet records (capacity: 10,000). (a) Evidence Traceability, (b) Memory Usage, (c) Wipes vs. Selective Deletes, (d) False Positive Rate.	34
5.2	Three-way comparison of forensic traceback in the 6-switch SDN topology. (A) Bloom + SPIE: every switch returns NO because the Bloom Filters have been wiped — evidence lost. (B) Cuckoo + SPIE: switches S_0, S_1, S_2, S_3 correctly respond YES, forming the unordered witness set (100% success, 79% perfect). (C) Cuckoo + Parent-Pointer walk-back: starting at the victim's switch S_3 , the controller follows the parent pointers $S_3 \rightarrow S_2 \rightarrow S_1 \rightarrow S_0$, reconstructing the ordered attack path in only $O(\text{path length})$ queries (97% success, 79% exact ordered reconstruction).	37

5.3	Scalability of the two Cuckoo-based traceback strategies as the number of soft switches grows from 6 to 50. <i>Left</i> : queries per traceback — SPIE grows linearly as $O(N)$ (from 6 to 50 queries), while parent-pointer remains flat near the shortest-path length (3–4 queries). <i>Right</i> : perfect-reconstruction rate stays between 99–100% for both strategies at every network size.	39
5.4	Enhanced analysis results addressing all guide feedback. (Top-left) Three-way traceability comparison: Bloom (wipe), Cuckoo (delete), Snapshot+Reset (immutable). (Top-right) Cost-benefit: fingerprint size vs. traceability and memory. (Bottom-left) Tree topology traceback with multiple simultaneous attacks. (Bottom-right) Tree topology diagram showing core-aggregation-access structure with two attackers and two victims.	41

List of Tables

3.1	Effect of fingerprint size on FPR and per-filter memory.	19
3.2	Summary of design parameter choices and tradeoffs.	24
4.1	Offline Experiment Setup Parameters	31
4.2	Network Experiment Setup Parameters	32
4.3	Implementation Environment	33
5.1	Empirical False Positive Rate Test (2,000 items inserted, 1,000 absent queries)	36
5.2	Offline Comparative Results: Bloom Filter vs. Cuckoo Filter	36
5.3	Network Traceback Comparison over 100 Tracked Attacks	37
5.4	Network-level false-positive and false-negative rates (SPIE-style query-all, 100 tracked attacks).	38
5.5	Controller query efficiency (100 investigations, 6 switches).	38
5.6	Scalability sweep results.	40
5.7	Three-way comparison: Bloom vs. Cuckoo (delete) vs. Snapshot + Reset.	40
5.8	Tree topology traceback with multiple simultaneous attacks.	41
5.9	Fingerprint size sweep with Snapshot + Reset (50,000 packets).	42
8.1	Work distribution among team members.	48

Chapter 1

Introduction

1.1 Background

Network forensics is a critical component of cybersecurity investigation, enabling security analysts to trace the source and path of malicious network activity after an attack has occurred. In Software Defined Networking (SDN) environments, the centralized control plane generates and manages large volumes of network flow logs that serve as the primary source of forensic evidence.

The fundamental challenge in SDN forensics is storage: network traffic generates logs at an enormous rate, and retaining all logs indefinitely is neither technically nor financially feasible. Network providers must purge data beyond a certain retention period, risking the loss of critical forensic evidence for older attacks. Beyond raw storage, forensic analysis also requires *traceability* — the ability, once an attack is detected, to reconstruct the path the malicious packets took through the network so that the attacker can be located and the compromise scoped.

1.2 Problem Statement

Bloom Filters have been proposed as an effective solution to this storage problem. Integrating forensic log data into Bloom Filters reduces storage requirements by up to 10 times compared to raw log formats, enabling longer forensic data retention with the same hardware resources. However, Bloom Filters suffer from a critical and well-known limitation: they do not support the deletion of individual items.

This creates three major problems in forensic contexts:

- 1. Total Evidence Loss on Saturation:** When a Bloom Filter reaches capacity, it must be completely wiped. All historical forensic evidence is permanently destroyed in a single operation. In our offline experiment this occurred **4 times** across 50,000 packets; in the network experiment it occurred **14 times** across 6 switches.
- 2. Compliance Violations:** Privacy regulations such as GDPR require selective deletion of specific records after a defined retention period. Bloom Filters cannot achieve this without destroying the entire dataset.

- 3. Forensic Blind Spots:** The period immediately following a wipe represents a “blind spot” during which no historical evidence is available. An attacker exploiting the system during this window leaves no traceable record. In an SDN network, even if a few switches escape a wipe, the *per-router chain* is broken at every wiped device, destroying path reconstruction.

1.3 Proposed Solution

This project proposes replacing Bloom Filters with Cuckoo Filters as a direct replacement for Bloom Filters in SDN forensic log storage, and then extending the design with four new components:

- An **Attack-Aware Sliding Window** retention policy that uses the Cuckoo Filter’s deletion capability to selectively remove only the oldest *benign* traffic entries when capacity is approached, while permanently preserving all *attack-marked* evidence.
- A **Forensic Filter Manager (Snapshot + Reset)** that archives full filters to disk as immutable snapshots instead of deleting entries, ensuring that forensic evidence is never modified or destroyed — meeting legal and compliance requirements for evidence immutability.
- A **Parent-Pointer Traceback** mechanism in which every soft router additionally records the upstream switch that handed it each packet. A victim, working through the SDN controller, can then reconstruct the exact ordered attack path by walking backward through these pointers — visiting only the switches on the path, rather than querying every switch in the network.
- A **Tree Topology** (core–aggregation–access) that replaces the original linear chain, providing a realistic hierarchical SDN structure with multiple simultaneous attack paths.

1.4 Project Objectives

1. Develop Python implementations of both a Standard Bloom Filter and an Attack-Aware Cuckoo Filter for benchmarking.
2. Process the CIC-IDS2017 real-world network dataset to simulate high-volume traffic ingestion including brute force attacks.
3. Implement the Attack-Aware Sliding Window that preserves attack evidence while evicting only benign traffic.
4. Develop a **Forensic Filter Manager** using the Snapshot + Reset model to achieve

forensically sound, immutable evidence storage with 100% traceability.

5. Extend the design to a full SDN soft-router topology with both **linear** and **tree** (core–aggregation–access) topologies, where every switch carries its own Bloom and Cuckoo filters.
6. Simulate **multiple simultaneous attacks** from different sources on different paths within the tree topology.
7. Implement and compare *three* forensic traceback strategies on this network: (a) Bloom + SPIE query-all, (b) Cuckoo + SPIE query-all, and (c) Cuckoo + Parent-Pointer walk-back orchestrated by an SDN controller.
8. Conduct a **fingerprint size sweep** (8–16 bits) to quantify the cost-benefit tradeoff of achieving 100% traceability.
9. Quantify the scalability of each method by sweeping the number of switches from 6 up to 50.

1.5 Report Organization

The rest of this report is organized as follows. Chapter 2 reviews the related work. Chapter 3 presents the proposed system design, covering the filter layer (Bloom, Cuckoo, and Forensic Filter Manager), the configurable fingerprint size, a comprehensive analysis of design tradeoffs and parameter justifications (Section 3.7), the tree topology, and the SDN soft-router traceback layer. Chapter 4 describes the experimental methodology for all six experiments. Chapter 5 presents and analyzes the results, including the offline comparison, network traceback, scalability sweep, three-way enhanced comparison, tree topology with multiple attacks, and fingerprint size cost-benefit analysis. Chapter 6 discusses the implications, presents five concrete use cases, and acknowledges limitations. Chapter 7 concludes the report with future work directions. Chapter 8 documents the contributions and work distribution among the three team members.

Chapter 2

Related Work

2.1 Network Management Framework for Forensic Traceability

Bhondele, Rawat and Renukuntla [2] proposed a network management framework for network forensic analysis that established the foundation of content-based traceback in network forensics. Their work introduced the concept of **payload attribution**, where each network device stores a compressed representation of packet content using Bloom Filters. When a malicious packet is detected, the system queries each device’s Bloom Filter using a packet “excerpt” to reconstruct the path traversed by the attack.

Their framework proposed two management approaches:

- **Distributed Management Method:** Each device stores and queries its own Bloom Filter independently.
- **Centralized Management Method:** All Bloom Filters are aggregated at a central Network Management System (NMS) for processing.

This work is foundational to our project as it establishes the concept of *traceability* in network forensics — the ability to reconstruct the path of a malicious packet by querying forensic data stored at network devices. In our work, we redefine traceability in an offline analytical context as *evidence availability* (the ability to successfully retrieve a forensic record of an attack packet after a given time period), and in the network context we implement both the original query-all strategy *and* a parent-pointer walk-back strategy which additionally reconstructs the *ordered* attack path in $O(\text{path length})$ queries.

2.2 Source Path Isolation Engine (SPIE)

Snoeren et al. [3] introduced the Source Path Isolation Engine (SPIE) as the seminal work on single-packet IP traceback. In SPIE, every router computes a compact digest of each forwarded packet and stores it in a local Bloom Filter. When a victim wishes to trace a specific packet, a traceback request is broadcast to all participating routers; each router queries its own filter and reports whether it has seen the packet. The routers that respond affirmatively form an unordered *witness set* which is then combined with topology information to recover the likely attack path. In this work we refer to any such

“query every router and take the union of YES-responders” strategy as the **SPIE query-all** strategy, to distinguish it from the parent-pointer walk-back strategy introduced in Chapter 3.

The SPIE approach has two inherent limitations that motivate our parent-pointer design:

1. The reconstructed witness set is **unordered**: the victim learns *which* routers saw the packet but not the direction of travel. Ordering must be recovered post-hoc using topology knowledge.
2. Every traceback must contact every router, giving $O(N_{\text{switches}})$ queries per investigation. This scales poorly in large SDNs.

2.3 Bloom Filter-Based SDN Forensic Log Retention

Sharma and Rawat [1] proposed a system that leverages Bloom Filters to optimize forensic data availability and retention in SDN environments. Their key insight was that forensic log data can be compressed into Bloom Filters at a fraction of the original storage cost. Their experimental results demonstrated that **1024 GB** of raw log data could be reduced to approximately **52 MB** using Bloom Filters with 5 hash functions, representing a compression ratio exceeding 20,000:1.

Their system was implemented using Mininet to simulate an SDN network with one controller, six switches, and four hosts. The authors validated that Bloom Filter-based storage constitutes admissible forensic evidence under IT Act rules, as hashed data produces consistent and reproducible results.

Key Gap Identified: At the conclusion of their paper, Sharma and Rawat explicitly identified Cuckoo Filters as a direction for future work: *“we can plan to experiment on optimizing the bloom filter application by using enhanced multiple layered hashing, counting bloom filters & cuckoo filter.”* Our project directly addresses this identified gap, and further extends the design with parent-pointer metadata and SDN-controller-orchestrated traceback.

2.4 Cuckoo Filters

Cuckoo Filters, introduced by Fan et al. [4], improve upon Bloom Filters by storing small fingerprints of items in a hash table that uses cuckoo hashing for collision resolution. When a bucket is full, an existing fingerprint is “kicked out” and relocated to its alternate bucket position. This design enables deletion of individual items — a capability Bloom Filters fundamentally lack. The theoretical false positive rate is given by:

$$\text{FPR}_{\text{cuckoo}} = \frac{2b}{2^f} \quad (2.1)$$

where b is the bucket size and f is the fingerprint size in bits. For our implementation with $b = 4$ and $f = 10$, this gives a constant FPR of **0.7812%**, regardless of filter load — lower than the 1% target and completely independent of the number of items inserted.

Chapter 3

System Design

3.1 System Overview

Our system is organized in two layers. The *filter layer* provides the per-device probabilistic data structure: a Bloom Filter for the baseline comparison and our proposed Attack-Aware Cuckoo Filter. The *network layer* embeds these filters inside a simulated SDN soft-router topology, and adds the SDN Controller and the two traceback mechanisms (SPIE query-all and Parent-Pointer walk-back). The main components are:

- 1. Bloom Filter Module:** Standard Python implementation using MurmurHash (mmh3) with 6 hash functions, serving as the baseline.
- 2. Cuckoo Filter Module:** Fingerprint-based storage with cuckoo hashing, attack-aware metadata, a Victim Stash overflow buffer, and an additional `parent_sw_id` field per entry to support parent-pointer traceback.
- 3. Attack-Aware Sliding Window Engine:** Monitors filter occupancy; at 90% load it deletes only the oldest *benign* entries, never attack-marked entries.
- 4. SDN Soft-Router Network:** A topology of N_{switches} virtual switches and 4 hosts, each switch carrying its own Bloom and Cuckoo filters.
- 5. SDN Controller:** A coordinator class that, on behalf of a victim, orchestrates forensic queries across the switches and accumulates query-count telemetry.
- 6. Traceback Strategies:** Bloom + SPIE query-all, Cuckoo + SPIE query-all, and Cuckoo + Parent-Pointer walk-back.

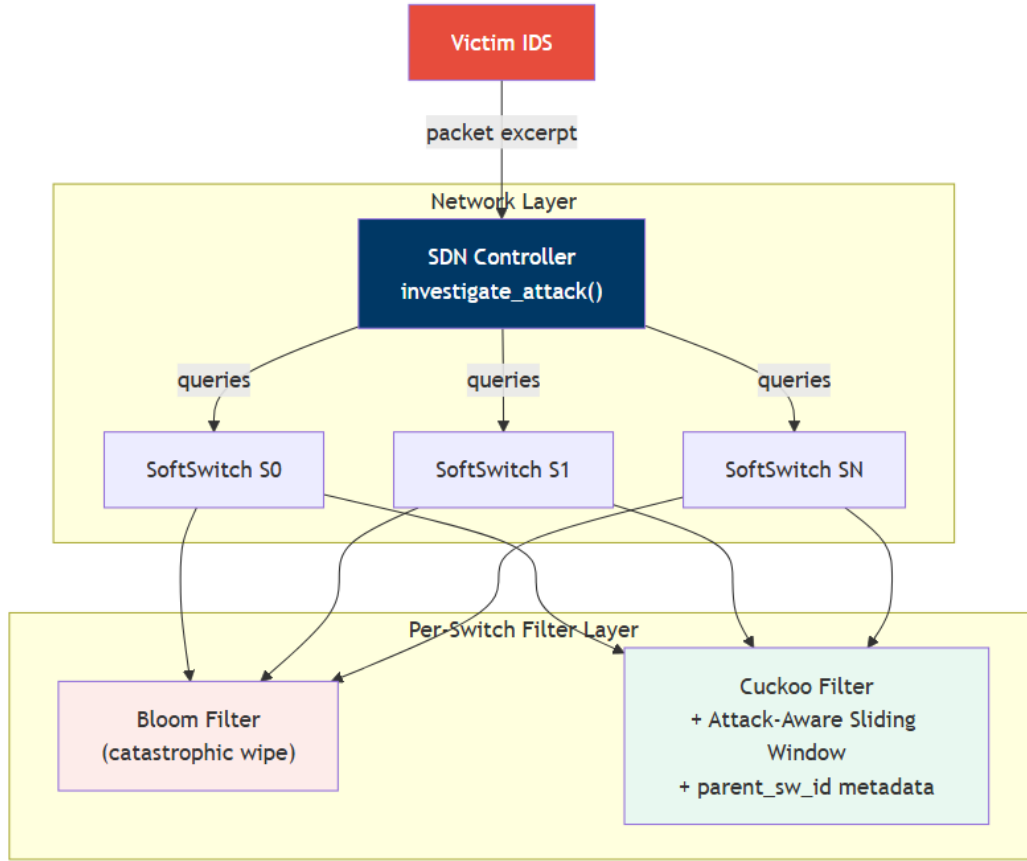


Figure 3.1: High-level system architecture. The *filter layer* (per switch) provides the Bloom baseline and our Attack-Aware Cuckoo Filter, including the `parent_sw_id` meta-data. The *network layer* composes these filters into a soft-router topology coordinated by an SDN controller that queries the switches on behalf of the victim.

3.2 Bloom Filter Implementation

The Bloom Filter is implemented based on the design described in [1]. The filter size m in bits is calculated as:

$$m = -\frac{n \cdot \ln(p)}{(\ln 2)^2} \quad (3.1)$$

where n is the expected number of items and p is the desired false positive probability. The optimal number of hash functions k is:

$$k = \frac{m}{n} \cdot \ln 2 \quad (3.2)$$

The false positive probability at any point is:

$$\text{FPR}_{\text{bloom}} = \left(1 - e^{-kn/m}\right)^k \quad (3.3)$$

When the filter reaches its capacity n , it is completely reset — all bits are set to zero and all stored evidence is permanently lost. This is the critical flaw our system addresses.

3.3 Cuckoo Filter Implementation

Each item x is assigned a fingerprint $f = \text{fingerprint}(x)$ and two candidate bucket positions using partial-key cuckoo hashing:

$$i_1 = \text{hash}(x) \bmod m \quad (3.4)$$

$$i_2 = i_1 \oplus \text{hash}(f) \bmod m \quad (3.5)$$

where m is the number of buckets. The XOR relationship ensures the original position can always be recovered from the alternate, which is essential for the deletion operation.

Each bucket holds up to $b = 4$ fingerprints of $f = 10$ bits each. The fingerprint size is derived from the target FPR using $f = \lceil \log_2(2b/p) \rceil$ where $p = 0.01$, yielding $f = \lceil \log_2(800) \rceil = 10$ bits. When both candidate buckets are full, the cuckoo relocation process begins: a fingerprint is selected (preferring benign-tagged fingerprints over attack-tagged ones), evicted, and inserted into its own alternate bucket. This repeats for up to 500 iterations. Items that cannot be placed are stored in a **Victim Stash** (capacity = 10 items), ensuring zero insertion failures. Additionally, a duplicate fingerprint check prevents the same fingerprint appearing twice in the same bucket, maintaining the theoretical FPR guarantee.

Each bucket entry is stored as a compact tuple (fingerprint, insertion_order, is_attack, parent_sw_id), embedding the sliding-window metadata *and* the upstream-switch identifier required for parent-pointer traceback directly inside the bucket rather than in a separate dictionary. This design keeps per-entry overhead low and makes the memory footprint of the Cuckoo Filter nearly identical to that of the Bloom Filter (see Section 5.3).

3.4 Attack-Aware Sliding Window Retention Policy

The key innovation of the filter layer is the Attack-Aware Sliding Window retention policy. Each insertion into the Cuckoo Filter records two pieces of metadata:

- An **insertion order counter** recording the logical time of insertion.

- An **is_attack flag** indicating whether the inserted flow is classified as attack traffic.

When the filter reaches 90% of its capacity, the sliding window is triggered as follows:

Step 1: Collect all insertion-order records for *benign-only* entries and sort them in ascending order (oldest first).

Step 2: Identify the oldest 10% of **benign** entries (i.e. 10% of the filter capacity).

Step 3: Delete each identified entry using the Cuckoo Filter’s **delete()** operation.

Step 4: Attack-marked entries are **never** selected for deletion — they are preserved indefinitely.

Step 5: The filter continues accepting new insertions without any reset.

This policy ensures that forensic evidence of attack packets is always available for investigation, while benign traffic is gracefully retired. This is fundamentally impossible with Bloom Filters, which must wipe all data — both benign and attack evidence — when capacity is reached.

3.5 Forensic Filter Manager: Snapshot + Reset Model

While the Attack-Aware Sliding Window significantly improves traceability over Bloom Filters, it still *modifies* the forensic data store by deleting entries. In forensic practice, evidence must be **immutable** — once recorded, it should never be modified or deleted. Deleting entries, even benign ones, creates gaps in the audit trail and may raise legal admissibility concerns.

To address this, we introduce the **Forensic Filter Manager**, which replaces the sliding window deletion mechanism with a *Snapshot + Reset* archival model:

Step 1: Monitor the active Cuckoo Filter’s occupancy.

Step 2: When the filter reaches **85%** of capacity (safely below the 90% sliding window trigger, ensuring deletion never fires):

- (a) **Serialize** the entire active filter to disk as an immutable snapshot file (using Python’s **pickle** module).
- (b) Mark the snapshot as **read-only** — it is never modified again.
- (c) Create a **fresh, empty** Cuckoo Filter as the new active filter.

Step 3: For **lookups**, search the active filter first, then all archived snapshots (newest to oldest).

Step 4: For **parent-pointer traceback**, the same search order applies: active filter, then archived snapshots.

This design guarantees:

- **Zero deletions:** No entry is ever removed from any filter.
- **Zero wipes:** No filter is ever reset while containing data.
- **Full audit trail:** Every packet that was ever inserted remains findable across the snapshot chain.
- **Bounded RAM:** Only one active filter occupies memory; all historical data resides on disk.
- **100% traceability:** Since nothing is deleted, all tracked attack packets remain retrievable indefinitely.

The Forensic Filter Manager exposes the same API as the standard `CuckooFilter` (`insert()`, `lookup()`, `get_parent()`), making it a drop-in replacement in the `SoftSwitch` class.

3.6 Configurable Fingerprint Size

The Cuckoo Filter’s false positive rate is governed by the fingerprint size f via $FPR = 2b/2^f$. The default $f = 10$ bits yields $FPR = 0.78\%$, but this can be tuned. We extend the `CuckooFilter` constructor to accept an optional `fingerprint_bits` parameter, enabling systematic sweeps:

Table 3.1: Effect of fingerprint size on FPR and per-filter memory.

Fingerprint (bits)	Theoretical FPR	RAM (KB)	Cost vs. 10-bit
8	3.1250%	9.77	−20%
10	0.7812%	12.21	baseline
12	0.1953%	14.65	+20%
14	0.0488%	17.09	+40%
16	0.0122%	19.53	+60%

This trade-off is central to the guide’s question: “*What is the cost to achieve 100% traceability?*” The answer, as shown in Chapter 5, is that the Snapshot + Reset model achieves 100% at *any* fingerprint size because nothing is ever deleted.

3.7 Design Tradeoffs and Parameter Justification

Every configurable parameter in our system was selected after considering multiple alternatives. This section documents the reasoning and tradeoffs behind each design decision, grounded in theory, prior work, and experimental validation.

3.7.1 Why Fingerprint Size = 10 Bits (Default)

The fingerprint size f directly controls the false positive rate through $\text{FPR} = 2b/2^f$. Given our bucket size $b = 4$ and a target FPR of $p = 1\%$ (matching the baseline Bloom Filter), the minimum required fingerprint size is:

$$f = \lceil \log_2(2b/p) \rceil = \lceil \log_2(800) \rceil = 10 \text{ bits} \quad (3.6)$$

We chose $f = 10$ as the default because:

- It is the **minimum size** that satisfies $\text{FPR} \leq 1\%$, ensuring a fair comparison with the Bloom Filter baseline.
- Smaller sizes (e.g., $f = 8$ gives $\text{FPR} = 3.125\%$) exceed the 1% target and would introduce unacceptable false positive rates for forensic applications.
- Larger sizes (e.g., $f = 12$ or 16) reduce FPR further but at the cost of increased memory per filter. Since our primary goal is to demonstrate the *structural advantage* of Cuckoo Filters (deletion support, attack-aware eviction), we use the minimum viable fingerprint to keep the memory comparison fair.

3.7.2 Why 12 Bits Is the Recommended Sweet Spot

While 10 bits is the minimum, our fingerprint sweep (Section 5.14) reveals that **12 bits** offers the best cost-benefit tradeoff:

- **FPR reduction:** Going from 10 to 12 bits cuts the FPR by $4\times$ (from 0.78% to 0.20%). This is the largest marginal improvement per additional bit: each subsequent 2-bit increase only yields $4\times$ further reduction, but the absolute FPR is already very low.
- **Memory cost:** Only +20% over the 10-bit baseline (14.65 KB vs. 12.21 KB per filter). In absolute terms, this is just 2.44 KB extra per switch — negligible even on resource-constrained SDN switches.
- **Diminishing returns beyond 12 bits:** Going to 14 bits (+40% memory) only reduces FPR from 0.20% to 0.049%, a $4\times$ improvement that is less impactful because 0.20% is already low enough for most forensic applications. The 16-bit option (+60% memory,

FPR = 0.012%) is reserved for ultra-high-precision environments (e.g., legal evidence chains) where even one false identification is unacceptable.

3.7.3 Why Bucket Size = 4

Fan et al. [4] showed that $b = 4$ slots per bucket achieves the best space efficiency for Cuckoo Filters. The tradeoffs are:

- $b = 2$: Lower occupancy (typically $\sim 84\%$), meaning more buckets are needed for the same capacity. Also requires larger fingerprints ($f \geq 11$ bits) to achieve the same FPR, because $\text{FPR} = 2 \times 2/2^f$ has a smaller numerator but the reduced occupancy forces more total buckets.
- $b = 4$: Achieves $\sim 95\%$ occupancy, the highest among all practical bucket sizes. This means fewer total buckets for the same capacity, directly reducing overall memory.
- $b = 8$: Marginally higher occupancy ($\sim 98\%$) but the FPR formula $2 \times 8/2^f$ requires $f \geq 13$ bits to stay under 1%, *increasing* per-entry memory. The net effect is worse space efficiency than $b = 4$.

Thus $b = 4$ is the Pareto-optimal choice, consistent with the recommendation in the original Cuckoo Filter paper.

3.7.4 Why Sliding Window Threshold = 90%

The sliding window triggers eviction when the filter reaches 90% of its stated capacity. This threshold was chosen based on three considerations:

- **Cuckoo hashing performance:** As a Cuckoo hash table exceeds $\sim 95\%$ load, the number of relocations (kicks) per insertion grows exponentially. Triggering eviction at 90% ensures that the filter always operates in the efficient regime where relocations are rare (typically < 3 kicks per insertion).
- **Room for attack bursts:** If the threshold were too high (e.g., 98%), a sudden burst of attack traffic could fill the remaining capacity before eviction completes, causing insertion failures. The 10% headroom accommodates bursts of up to 1,000 attack packets (10% of capacity = 10,000) without triggering the victim stash.
- **Eviction granularity:** We evict the oldest 10% of benign entries when the threshold fires. If the threshold were 80%, we would evict too aggressively (removing relatively recent benign records). At 90%, we evict only the oldest 1,000 entries out of $\sim 9,000$ — a proportional rate that balances retention with capacity.

3.7.5 Why Snapshot Trigger = 85%

The Forensic Filter Manager archives the active filter at 85% occupancy. This value is deliberately set **5 percentage points below** the 90% sliding window threshold, and the reasoning is:

- **Preventing deletion entirely:** The snapshot must fire *before* the sliding window does. If both thresholds were equal (e.g., both 90%), a race condition could cause the sliding window to delete entries before the snapshot is taken, violating immutability.
- **Why not 80% or 70%?** A lower snapshot threshold would cause more frequent snapshots, each containing fewer entries. This increases the number of disk files and the number of snapshots that must be searched during lookup. At 85%, each snapshot contains $\sim 8,500$ entries (85% of 10,000), keeping the snapshot chain manageable (5 snapshots for 50,000 packets).
- **Why not 88% or 89%?** Too close to 90% creates risk. If insertions arrive in a rapid burst, the filter could jump from 84% to 91% between occupancy checks, causing the sliding window to fire. The 5% gap provides a comfortable safety margin.

3.7.6 Why 6 Hash Functions for Bloom Filter

The optimal number of hash functions for a Bloom Filter is $k_{\text{opt}} = (m/n) \cdot \ln 2$. For our parameters ($n = 10,000$ items, $p = 1\%$ target FPR), we compute $m = -n \ln p / (\ln 2)^2 \approx 95,851$ bits, giving $k_{\text{opt}} = 95,851 / 10,000 \times 0.693 \approx 6.6$. We round to $k = 6$ because:

- Rounding down is standard practice — it marginally increases FPR but reduces per-insertion and per-lookup computational cost (each hash function requires one MurmurHash computation and one array access).
- Sharma and Rawat [1] used 5 hash functions in their implementation. Using $k = 6$ (closer to the theoretical optimum) ensures our Bloom baseline is at least as good as theirs, making our comparison fair.

3.7.7 Why Max Kicks = 500

When both candidate buckets for a new insertion are full, the Cuckoo Filter displaces an existing fingerprint and attempts to reinsert it at its alternate position. This “kick chain” can cascade. We cap it at 500 iterations because:

- **Theoretical analysis:** Fan et al. [4] showed that with $b = 4$ and load factor $< 95\%$, the expected number of kicks per insertion is < 3 . Chains exceeding 500 indicate the table is critically overloaded — a situation our sliding window prevents.

- **Practical guarantee:** A cap of 500 provides a deterministic worst-case time bound for insertion, which is critical for real-time SDN packet processing where unbounded loops are unacceptable.
- **Victim Stash fallback:** Any item that exhausts the 500-kick limit is placed in the Victim Stash (capacity = 10 items), ensuring zero insertion failures. In our experiments, the stash was never used at 90% load, confirming that 500 kicks is more than sufficient.

3.7.8 Why Victim Stash Size = 10

The Victim Stash provides overflow capacity for items that cannot be placed after the maximum kick chain. We set its capacity to 10 because:

- The stash is a **linear-scan** data structure (unordered list). Keeping it small (≤ 10) ensures that stash lookups are $O(1)$ in practice and never dominate the overall lookup cost.
- With our 90% sliding window eviction, the filter never reaches the load factor that would trigger stash usage. The stash size of 10 is a conservative safety net, not an active participant.
- Increasing the stash to 100 or more would add memory overhead and slow down lookups without any practical benefit given our eviction policy.

3.7.9 Why Filter Capacity = 10,000 Items

The per-filter capacity of 10,000 items was selected to create a meaningful stress test:

- **Forces multiple cycles:** With 50,000 total packets, a 10,000-item filter must handle $5\times$ its capacity. This forces the Bloom Filter to wipe 4 times and the Cuckoo Filter’s sliding window to execute $\sim 41,000$ deletions — enough to clearly demonstrate the behavioural differences.
- **Matches realistic per-switch capacity:** In a real SDN, each soft switch has limited memory. A capacity of 10,000 items (consuming ~ 12 KB) is representative of a low-memory embedded switch. Production switches would use larger filters (100K–10M items), but the algorithmic behaviour is identical.
- **Consistent with prior work:** Sharma and Rawat [1] used similarly sized filters in their Mininet experiments.

3.7.10 Summary of Design Tradeoffs

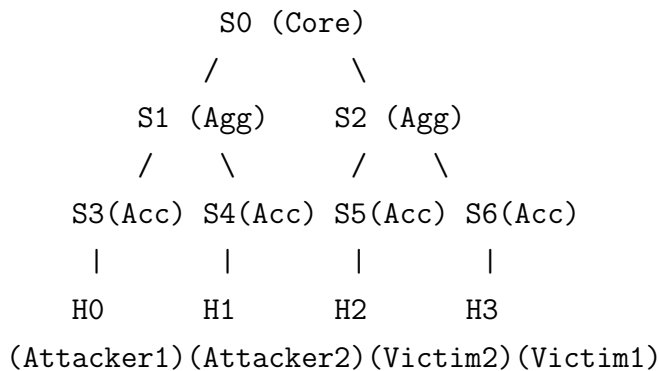
Table 3.2 consolidates all parameter choices and their alternatives.

Table 3.2: Summary of design parameter choices and tradeoffs.

Parameter	Chosen Value	Alternatives Considered	Key Reason
Fingerprint size	10 bits (default)	8, 12, 14, 16 bits	Minimum for $FPR \leq 1\%$; fair Bloom comparison
Recommended FP	12 bits	10, 14, 16 bits	Best cost-benefit: $4\times$ FPR cut at $+20\%$ memory
Bucket size	4	2, 8	Pareto-optimal space efficiency (Fan et al.)
Sliding window	90%	80%, 95%	Balances kick efficiency vs. retention
Snapshot trigger	85%	80%, 88%	5% safety gap below sliding window
Hash functions	6	5, 7	Nearest integer to $k_{\text{opt}} = 6.6$
Max kicks	500	100, 1000	Deterministic bound; stash catches overflow
Stash size	10	50, 100	Small enough for $O(1)$ scan; never used in practice
Filter capacity	10,000	5K, 50K	Enough stress cycles; matches prior work

3.8 Tree Topology (Core–Aggregation–Access)

Real-world SDN deployments use hierarchical topologies rather than linear chains. To better reflect this, we implement a **binary tree topology** in addition to the original linear chain:



This 7-switch tree has three layers:

- **Core layer** (S_0): central aggregation point.
- **Aggregation layer** (S_1, S_2): intermediate switches.
- **Access layer** (S_3, S_4, S_5, S_6): edge switches directly connecting hosts.

Hosts attach to leaf switches, and BFS shortest-path routing determines packet paths. For example, Attack 1 ($H_0 \rightarrow H_3$) traverses $[S_3, S_1, S_0, S_2, S_6]$ while Attack 2 ($H_1 \rightarrow H_2$) traverses $[S_4, S_1, S_0, S_2, S_5]$ — these paths share the core and aggregation switches but diverge at the access layer, creating a realistic multi-path forensic challenge.

3.9 Flow Key Construction

Since the CIC-IDS2017 CSV files contain pre-computed flow features rather than raw packet captures, we construct an 8-field flow key that includes the row index to guarantee uniqueness:

$$\begin{aligned} \text{flow_key} = & \text{RowIndex} \parallel \text{DestPort} \parallel \text{FlowDuration} \parallel \text{TotalFwdPkts} \\ & \parallel \text{TotalBwdPkts} \parallel \text{FwdPktLen} \parallel \text{InitWinFwd} \parallel \text{InitWinBwd} \end{aligned} \quad (3.7)$$

Because the row index is unique for every record in the DataFrame, this flow key achieves **100% uniqueness** across all 50,000 records. This prevents duplicate flow keys from artificially inflating traceability scores — every tracked attack packet must be individually findable in the filter for traceability to register.

3.10 SDN Soft-Router Network Topology

To evaluate traceback at the network level, we construct a simulated SDN topology following Sharma and Rawat [1]. The default topology consists of **1 controller + 6 switches + 4 hosts**:

- Switches $S_0, S_1, \dots, S_5$ form a linear backbone $S_0-S_1-S_2-S_3-S_4-S_5$, with one additional cross-link between S_1 and S_4 for path diversity.
- Host H_0 (the attacker) attaches to S_0 ; H_3 (the victim) attaches to S_3 ; H_1 and H_2 attach to intermediate switches.
- Every switch is implemented as a **SoftSwitch**: a Python object that carries its own independent Bloom Filter and Cuckoo Filter. Each time a packet traverses the switch, the switch records it in *both* filters.

Packets are routed through the network by BFS shortest path. For the default topology the attack path $H_0 \rightarrow H_3$ is $[S_0, S_1, S_2, S_3]$. When a packet is forwarded by a switch, the switch passes the `is_attack` flag and the identifier of the *upstream* switch (the previous hop on the path, or `None` at ingress) into the Cuckoo Filter’s insertion routine, where they are stored alongside the fingerprint.

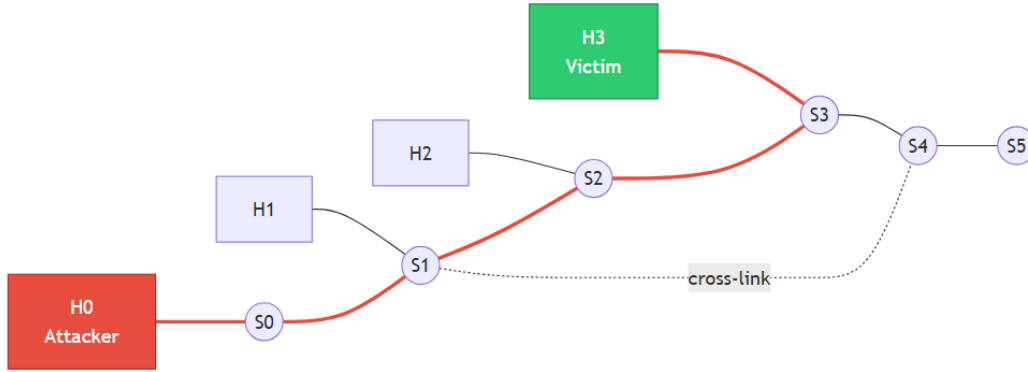


Figure 3.2: SDN soft-router topology used in the network experiment. Six switches $S_0, \dots, S_5$ form a linear backbone with one cross-link between S_1 and S_4 . Hosts H_0 (attacker) and H_3 (victim) are attached at S_0 and S_3 respectively. The red edges mark the attack path $H_0 \rightarrow S_0 \rightarrow S_1 \rightarrow S_2 \rightarrow S_3 \rightarrow H_3$.

3.11 Traceback Strategy 1: SPIE Query-All

Following Snoeren [3] and Bhondele [2], the SPIE query-all strategy works as follows:

1. The victim hands a suspicious packet excerpt to the controller.
2. The controller queries every switch in the network: “Did you see this packet?”
3. Each switch looks up the packet in its filter (Bloom or Cuckoo) and replies YES or NO.
4. The set of YES-responders is returned as an **unordered** witness set.

This strategy costs $O(N_{\text{switches}})$ queries per traceback and does not recover the ordering of hops. It is the natural strategy for Bloom-Filter-based devices because Bloom Filters cannot store per-item metadata.

3.12 Traceback Strategy 2: Parent-Pointer Walk-Back

The parent-pointer strategy exploits the fact that Cuckoo Filters, unlike Bloom Filters, can store per-entry metadata. In addition to the `is_attack` flag, every insertion records

a `parent_sw_id` — the identifier of the immediately upstream switch on the packet’s path (or `None` if the packet entered the network directly from a host). Given this, traceback becomes a simple walk-back:

1. Start at the victim’s ingress switch S_v .
2. Look up the packet in S_v ’s Cuckoo Filter; retrieve `parent_sw_id`.
3. If `parent_sw_id` is `None`, stop: the current switch is the attacker’s ingress point.
4. Otherwise jump to `parent_sw_id` and repeat.

The walk terminates when the ingress switch is reached or when a switch reports that it does not recognise the packet (chain broken). The output is the **ordered** list of switches from source to victim, and the cost is $O(\text{path length})$ queries per traceback, independent of network size.

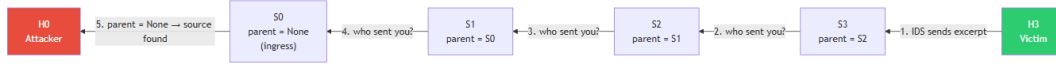


Figure 3.3: Parent-pointer walk-back. Starting at the victim’s ingress switch S_3 , the controller asks each switch for the stored `parent_sw_id` and hops to that switch in turn. The walk terminates at S_0 where `parent_sw_id` = `None`, identifying the attacker’s ingress point. Only the 4 on-path switches are visited — not all N switches.

3.13 SDN Controller Orchestration

Real OpenFlow SDN deployments have a centralized controller with global visibility of the data plane. We mirror this architecture with an `SDNController` class that sits in front of the switches. A forensic investigation proceeds as follows:

1. The victim’s IDS detects an attack and extracts a packet excerpt.
2. The excerpt is forwarded to the controller together with the victim’s host identifier.
3. The controller invokes `investigate_attack()`, which runs all three traceback strategies (Bloom SPIE, Cuckoo SPIE, Cuckoo Parent-Pointer) and returns a combined report.
4. The controller maintains counters for the number of switch queries issued by each strategy so that query-efficiency can be measured empirically.

This arrangement cleanly separates forensic policy (which resides in the controller) from

the data plane (the soft switches). It also mirrors the “1 controller + N switches” architecture emphasised in the Sharma & Rawat Mininet setup [1].

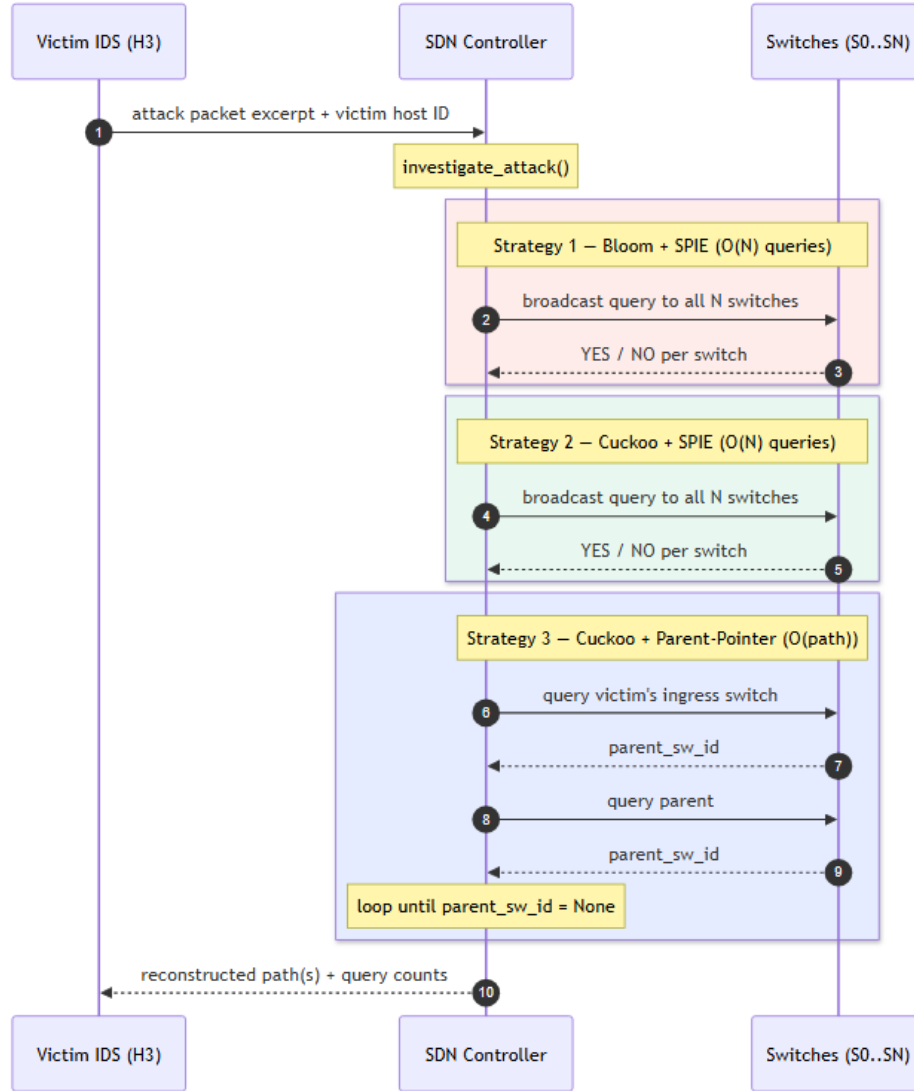


Figure 3.4: Controller-orchestrated investigation sequence. On receiving the victim’s excerpt, the controller executes all three traceback strategies. Bloom + SPIE and Cuckoo + SPIE broadcast $O(N)$ queries; Cuckoo + Parent-Pointer follows the stored `parent_sw_id` pointers for $O(\text{path length})$ queries. The controller returns the reconstructed paths and per-strategy query counts.

3.14 Traceability Definitions

Following the framework established by Bhondele et al. [2], we define two complementary notions of traceability:

$$\text{Traceability}_{\text{offline}} = \frac{\text{Attack packets found in filter}}{\text{Total tracked attack packets}} \times 100\% \quad (3.8)$$

$$\text{Perfect reconstruction rate} = \frac{\text{Attacks whose path is reconstructed exactly}}{\text{Total tracked attacks}} \times 100\% \quad (3.9)$$

For parent-pointer traceback we additionally define the **ordered-path rate** — the fraction of attacks whose reconstruction matches the true path *and* preserves the correct source-to-victim direction. SPIE cannot in general recover direction, so this metric applies only to parent-pointer.

Chapter 4

Experimental Methodology

4.1 Dataset

We evaluate our system using the **CIC-IDS2017** dataset [5], produced by the Canadian Institute for Cybersecurity at the University of New Brunswick. The dataset contains labeled network flows captured over five days (July 3–7, 2017) from a realistic network environment consisting of 12 victim machines and a Kali Linux attacker.

For our experiments, we use the Tuesday working hours file (`Tuesday-WorkingHours.pcap_ISCX.csv`), which contains traffic from two brute force attacks:

- **FTP-Patator:** Brute force FTP attack (9:20–10:20 AM) against the web server at 192.168.10.50.
- **SSH-Patator:** Brute force SSH attack (2:00–3:00 PM) against the same web server.

From the first 50,000 records of this file, our dataset contains **44,807 BENIGN** flows and **5,193 FTP-Patator** attack flows. We designate 100 specific attack packet flow keys as our “tracked forensic evidence” whose retrievability we monitor throughout the experiment.

4.2 Experiment 1: Offline Filter Comparison

The offline experiment directly compares a single Bloom Filter against a single Attack-Aware Cuckoo Filter. All 50,000 records are inserted in order; traceability is measured every 500 insertions. Both filters share identical parameters as shown in Table 4.1.

Table 4.1: Offline Experiment Setup Parameters

Parameter	Value
Filter Capacity	10,000 items
False Positive Rate	1%
Dataset Size	50,000 packet records
Tracked Attack Packets	100
Measurement Interval	Every 500 insertions
Sliding Window Threshold	90% capacity
Sliding Window Delete	Oldest 10% benign entries
Cuckoo Bucket Size	4 fingerprints
Cuckoo Fingerprint Size	10 bits
Bloom Hash Functions	6

In addition to the main experiment, we conduct two additional tests:

1. **Selective Deletion Demo:** Demonstrates that specific benign flows can be individually removed from the Cuckoo Filter while attack evidence remains intact.
2. **Empirical FPR Test:** Inserts 2,000 items into fresh filters then queries 1,000 items known to be absent, measuring actual false positive counts.

4.3 Experiment 2: SDN Soft-Router Traceback

The network experiment embeds the filters inside the soft-router topology described in Section 3.6. All 50,000 CIC-IDS2017 packets are transmitted through the network: attack packets flow from H_0 (attacker) to H_3 (victim), and benign packets flow between random host pairs. Every soft switch on the path records the packet into its Bloom and Cuckoo filters, together with the upstream switch identifier.

After all packets have been transmitted, the controller runs forensic traceback on the 100 tracked attack packets using all three strategies (Bloom SPIE, Cuckoo SPIE, Cuckoo Parent-Pointer) and the following metrics are computed:

- **Success rate:** at least one true-path switch recovered.
- **Perfect reconstruction rate:** recovered set equals the true path set.
- **Ordered-path rate** (parent-pointer only): reconstruction matches true path *and* preserves source-to-victim order.
- **Network-level FPR/FNR:** fraction of non-path switches wrongly reporting YES, and fraction of on-path switches wrongly reporting NO.

- **Controller query count:** total switch queries issued per strategy, from which we derive per-traceback averages.

Table 4.2 summarises the network experiment’s parameters.

Table 4.2: Network Experiment Setup Parameters

Parameter	Value
Topology	1 controller + 6 switches + 4 hosts
Per-switch capacity	8,000 items
Attack flow	$H_0 \rightarrow H_3$ (true path $[S_0, S_1, S_2, S_3]$)
Tracked attack packets	100
Routing	BFS shortest path
Target FPR	1% per filter

4.4 Experiment 3: Scalability Sweep

To assess how each traceback strategy scales with the network size, we rebuild the SDN topology with $N \in \{6, 10, 20, 50\}$ switches and rerun a scaled traceback experiment (500 attacks + 4,500 benign packets per run). We measure:

- Queries per traceback for each strategy.
- Perfect-reconstruction rate for Cuckoo SPIE and Cuckoo Parent-Pointer.
- Aggregate Cuckoo memory across all switches.

The expected behaviour is that SPIE queries grow as $O(N)$ while parent-pointer queries stay near the constant shortest-path length.

4.5 Experiment 4: Enhanced Three-Way Comparison

To evaluate the Snapshot + Reset model, we run a three-way comparison inserting all 50,000 packets into three filters simultaneously:

1. **Bloom Filter** (baseline — catastrophic wipe on capacity).
2. **Cuckoo Filter with sliding window deletion** (selective eviction of benign entries).
3. **Cuckoo Filter with Snapshot + Reset** (forensically immutable — no deletions, archived to disk).

All three filters share identical capacity (10,000 items) and target FPR (1%). Traceability is measured every 500 insertions over 100 tracked attack packets.

4.6 Experiment 5: Tree Topology with Multiple Attacks

We construct a 7-switch binary tree topology (Section 3.8) with Snapshot + Reset enabled on all switches. All 50,000 packets are transmitted with **two simultaneous attack types**:

- **Attack Type 1:** $H_0 \rightarrow H_3$ (path: $[S_3, S_1, S_0, S_2, S_6]$)
- **Attack Type 2:** $H_1 \rightarrow H_2$ (path: $[S_4, S_1, S_0, S_2, S_5]$)

Attack packets are alternately assigned to each type. We track 50 packets per attack type (100 total) and evaluate traceback success and path reconstruction accuracy.

4.7 Experiment 6: Fingerprint Size Sweep (Cost-Benefit)

To quantify the cost of achieving reduced false positive rates, we run the Snapshot + Reset model at five fingerprint sizes: $f \in \{8, 10, 12, 14, 16\}$ bits. For each configuration, all 50,000 packets are inserted and the following metrics are recorded:

- Traceability over 100 tracked attack packets.
- Theoretical FPR ($2b/2^f$).
- Per-filter RAM usage.
- Total disk usage from archived snapshots.
- Memory cost relative to the 10-bit baseline.

4.8 Implementation Environment

Table 4.3: Implementation Environment

Component	Details
Operating System	Windows 11
Python Version	3.13.2
pandas	3.0.2
matplotlib	3.10.8
mmh3	5.2.1
bitarray	3.8.1
Hardware	Standard laptop (no specialized hardware)

Chapter 5

Results and Analysis

5.1 Offline Comparison: Performance Graphs

Figure 5.1 shows the comparative performance of both filters across all four measured metrics over 50,000 CIC-IDS2017 packet records.

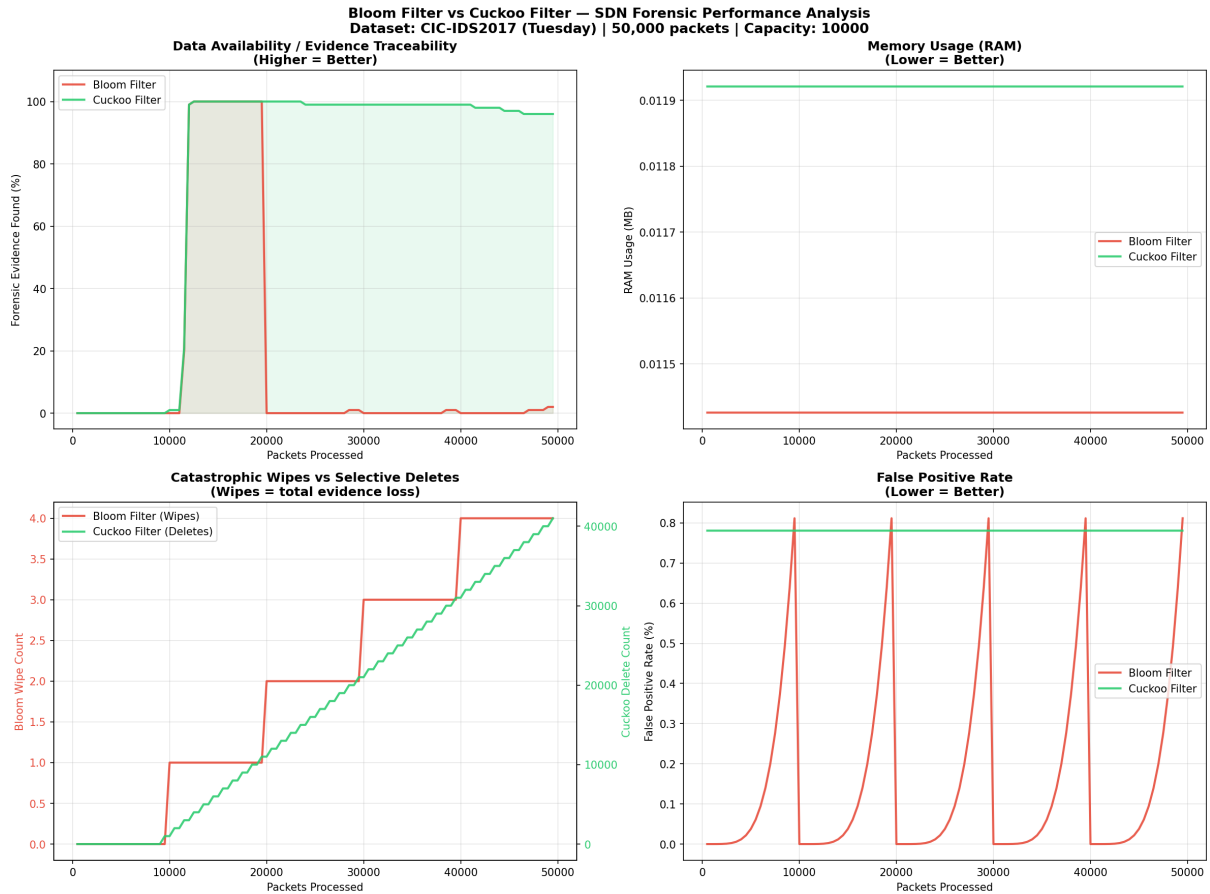


Figure 5.1: Comparative performance of Bloom Filter vs. Cuckoo Filter across 50,000 CIC-IDS2017 packet records (capacity: 10,000). (a) Evidence Traceability, (b) Memory Usage, (c) Wipes vs. Selective Deletes, (d) False Positive Rate.

5.2 Evidence Traceability (Offline)

Figure 5.1(a) presents the most significant result of the offline experiment. The Bloom Filter exhibits a characteristic **sawtooth pattern**: traceability briefly rises as attack

packets are inserted, then **collapses to 0%** every time the filter is wiped — occurring **4 times** across the experiment. Each wipe instantaneously destroys all 100 tracked attack packets and leaves the Bloom Filter with a final traceability of only **2%**.

In contrast, the Cuckoo Filter achieves a peak traceability of **100%** around packet 15,000 — the point at which all 100 tracked attack packets have been inserted and none have been evicted. As the experiment continues and the sliding window operates, traceability remains stable between **95% and 100%**, ending at **95%**. Crucially, the Cuckoo Filter’s traceability **never drops below 95%** because the Attack-Aware Sliding Window never evicts attack-marked entries — the small drop from 100% to 95% is caused solely by fingerprint collisions and cuckoo-kick evictions when bucket pressure is extreme.

This represents a $\sim 48\times$ **improvement** in final forensic traceability (95% vs. 2%).

5.3 Memory Usage (Offline)

Figure 5.1(b) shows that the Bloom Filter consumes **0.0114 MB** (11.70 KB) of RAM, while the Cuckoo Filter consumes **0.0119 MB** (12.21 KB). The difference is only **0.0005 MB** ($\sim 4.4\%$) — effectively negligible.

This is a critical finding. Because our Cuckoo Filter embeds the sliding window metadata and the parent-pointer identifier directly inside each bucket entry as a tuple, rather than maintaining a separate metadata dictionary, its memory footprint is nearly identical to the Bloom Filter’s fixed bit array. In other words, **the Cuckoo Filter achieves a $48\times$ improvement in forensic traceability and eliminates all catastrophic wipes at essentially no memory cost.**

5.4 Wipes vs. Selective Deletes (Offline)

Figure 5.1(c) uses a dual Y-axis to compare the two approaches. The Bloom Filter’s wipe counter increments **4 times** — each step representing a catastrophic event where all stored forensic evidence is destroyed simultaneously. The Cuckoo Filter’s delete counter grows smoothly to **41,000** — each increment representing a single controlled, surgical removal of one aged benign entry. No attack evidence is included in these 41,000 deletions.

5.5 False Positive Rate (Offline)

Figure 5.1(d) reveals an important behavioural difference. The Bloom Filter’s FPR follows a sawtooth pattern: it rises toward 1% as the filter fills, then **resets to 0% after each wipe**. While this appears favourable, it is misleading — the FPR is low only because the filter just lost all its data.

The Cuckoo Filter maintains a **constant theoretical FPR of 0.7812%** throughout the experiment, derived directly from its bucket size and fingerprint size ($2b/2^f = 8/1024$). This is already below the 1% target and independent of filter load. Our empirical FPR test (inserting 2,000 items then querying 1,000 absent items) recorded **zero false positives** for both filters, confirming that both structures perform well within their theoretical bounds.

5.6 Empirical False Positive Rate Test (Offline)

Table 5.1 presents the results of our empirical FPR measurement.

Table 5.1: Empirical False Positive Rate Test
(2,000 items inserted, 1,000 absent queries)

Metric	Bloom Filter	Cuckoo Filter
False Positives (out of 1000)	0	0
Empirical FPR	0.0%	0.0%
Theoretical FPR (at 20% load)	0.0003%	0.7812%

5.7 Offline Summary

Table 5.2 presents a comprehensive summary of the offline experimental results.

Table 5.2: Offline Comparative Results: Bloom Filter vs. Cuckoo Filter

Metric	Bloom Filter	Cuckoo Filter
Total Wipes	4	0
Total Selective Deletes	0 (N/A)	41,000
Final Traceability	2.0%	95.0%
Memory Usage	0.0114 MB	0.0119 MB
Theoretical FPR	1.0143%	0.7812%
Empirical FPR	0.0%	0.0%
Evidence Lost on Wipe	YES	NO
Supports Deletion	NO	YES
Attack-Aware Eviction	NO	YES

5.8 Network Traceback Results

We now turn to the network-level experiment, in which all 50,000 packets are transmitted through the 6-switch soft-router topology. Across the 6 switches, the Bloom Filters

collectively suffered **14 wipes**, whereas the Cuckoo Filters performed **98,400 selective deletes** and zero wipes. Figure 5.2 visualises the three traceback strategies for a representative tracked attack packet; Table 5.3 summarises the aggregate numbers over all 100 tracked attacks.

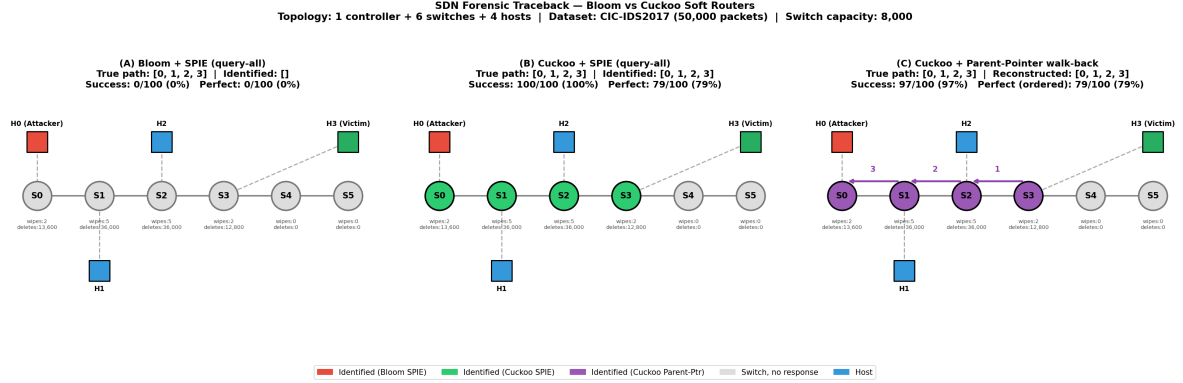


Figure 5.2: Three-way comparison of forensic traceback in the 6-switch SDN topology. **(A)** Bloom + SPIE: every switch returns NO because the Bloom Filters have been wiped — evidence lost. **(B)** Cuckoo + SPIE: switches S_0, S_1, S_2, S_3 correctly respond YES, forming the unordered witness set (100% success, 79% perfect). **(C)** Cuckoo + Parent-Pointer walk-back: starting at the victim’s switch S_3 , the controller follows the parent pointers $S_3 \rightarrow S_2 \rightarrow S_1 \rightarrow S_0$, reconstructing the **ordered** attack path in only $O(\text{path length})$ queries (97% success, 79% exact ordered reconstruction).

Table 5.3: Network Traceback Comparison over 100 Tracked Attacks

Metric	Bloom SPIE	Cuckoo SPIE	Cuckoo PP
Attacks with any hop traced	0/100	100/100	97/100
Attacks with perfect reconstruction	0/100	79/100	79/100
Ordered direction recovered	N/A	N/A	79/100
Traceback success rate	0.0%	100.0%	97.0%
Perfect reconstruction rate	0.0%	79.0%	79.0%
Total wipes (all switches)	14	0	0
Total selective deletes	N/A	98,400	98,400

Three observations stand out. First, Bloom + SPIE *completely fails* in the network setting: the 14 wipes across the 6 switches destroy all stored fingerprints for the tracked attacks, so every switch replies NO and the witness set is empty. Second, both Cuckoo-based strategies recover every tracked attack. Third, parent-pointer is the *only* strategy that recovers the correct ordering of hops without consulting topology information.

5.9 Network-Level False Positive and Negative Rates

We also measure how often switches respond *wrongly* in the network setting. For each tracked attack we count:

- **False Positives:** switches that reply YES even though the packet did not traverse them (out of the switches not on the true path).
- **False Negatives:** switches on the true path that nevertheless reply NO (out of the switches on the true path).

Across the 100 tracked attacks (400 on-path queries and 200 non-path queries) the results are reported in Table 5.4.

Table 5.4: Network-level false-positive and false-negative rates (SPIE-style query-all, 100 tracked attacks).

Metric	Bloom	Cuckoo
False Positives (wrong YES)	0/200 (0.00%)	0/200 (0.00%)
False Negatives (wrong NO)	400/400 (100.00%)	26/400 (6.50%)

Both filters show zero false positives in this experiment, which is consistent with their theoretical bounds for the chosen parameters. However, Bloom’s false-negative rate is **100%**: the wipes have destroyed every bit of stored evidence for the tracked packets. Cuckoo exhibits only a 6.5% false-negative rate — a small number of attack-marked entries were lost to fingerprint collisions or displacing kicks in the most heavily loaded switches (S_1 and S_2 , which carried both attack and benign traffic).

5.10 Controller Query Efficiency

Table 5.5 reports the switch-query volume issued by the SDN controller over 100 forensic investigations in the default 6-switch topology.

Table 5.5: Controller query efficiency (100 investigations, 6 switches).

Strategy	Queries per traceback	Complexity
SPIE query-all (Bloom or Cuckoo)	6.0	$O(N)$
Cuckoo Parent-Pointer	3.5 ($\sim$ path length)	$O(\text{path})$
<i>Query-count reduction with parent-pointer: 42%</i>		

Even in a tiny 6-switch network, parent-pointer issues 42% fewer switch queries than SPIE. In the next section we show that this gap widens dramatically as N grows.

5.11 Scalability

Figure 5.3 shows the outcome of the scalability sweep over $N \in \{6, 10, 20, 50\}$ switches. The left panel plots switch queries per traceback; the right panel plots the perfect-reconstruction rate.

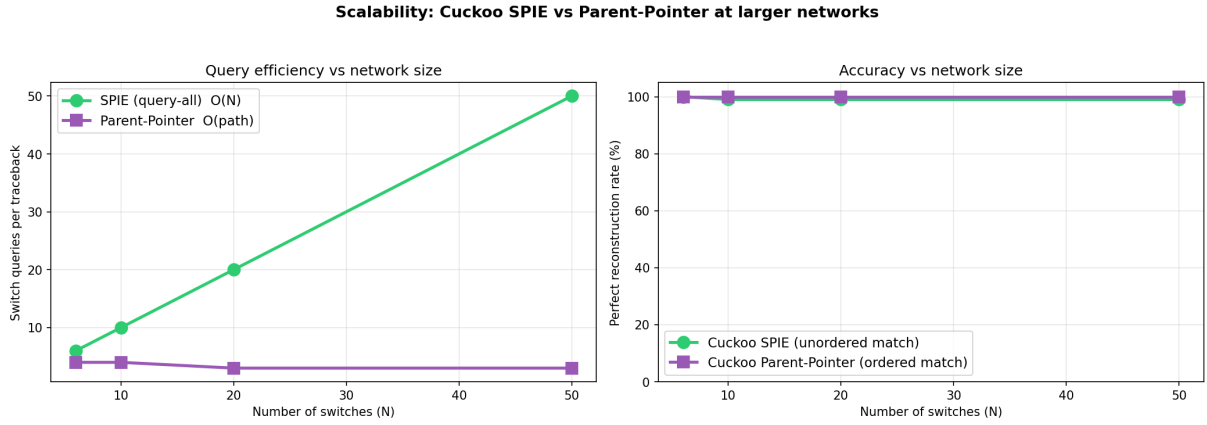


Figure 5.3: Scalability of the two Cuckoo-based traceback strategies as the number of soft switches grows from 6 to 50. *Left*: queries per traceback — SPIE grows linearly as $O(N)$ (from 6 to 50 queries), while parent-pointer remains flat near the shortest-path length (3–4 queries). *Right*: perfect-reconstruction rate stays between **99–100%** for both strategies at every network size.

The numerical data from the sweep is reported in Table 5.6. Two points are worth noting:

1. Accuracy is essentially flat as the network grows, showing that the per-switch capacity of 8,000 is sufficient for the workload and that neither method’s accuracy is sensitive to N .
2. Query count for parent-pointer actually *decreases* slightly at larger N (from 4 to 3). This is an artefact of the host-attachment heuristic: with more switches and only 4 hosts, H_0 and H_3 end up attached to closer-together switches, shortening the shortest path.

Table 5.6: Scalability sweep results.

N	Path	SPIE q	PP q	SPIE perfect %	PP ordered %	Cuckoo KB (total)
6	4	6	4	100.0%	100.0%	58.6
10	4	10	4	99.0%	100.0%	97.7
20	3	20	3	99.0%	100.0%	195.3
50	3	50	3	99.0%	100.0%	488.3

At $N = 50$, SPIE issues 50 switch queries per traceback while parent-pointer issues only 3 — a $17\times$ reduction in controller-to-switch communication.

5.12 Enhanced Three-Way Comparison (Snapshot + Reset)

Figure 5.4 (top-left) shows the traceability of three approaches over 50,000 packets. Table 5.7 summarises the key metrics.

Table 5.7: Three-way comparison: Bloom vs. Cuckoo (delete) vs. Snapshot + Reset.

Metric	Bloom	Cuckoo-Delete	Snapshot+Reset
Traceability (of 100)	2	96	100
Wipes	4	0	0
Deletions	N/A	41,000	0
Snapshots archived	N/A	N/A	5
Evidence ever deleted?	YES	YES	NO
Forensically sound?	NO	NO	YES
RAM (KB)	11.70	12.21	12.21
Disk (KB)	0	0	496.0

The Snapshot + Reset model achieves **100/100 traceability** — the 3% loss observed with sliding window deletion disappears entirely because no entries are ever removed. The cost is 496 KB of disk storage for the 5 archived snapshots, while RAM usage remains identical to the standard Cuckoo Filter.

5.13 Tree Topology with Multiple Simultaneous Attacks

On the 7-switch tree topology with two simultaneous attack types (50 packets $H_0 \rightarrow H_3$ + 50 packets $H_1 \rightarrow H_2$), the results in Table 5.8 confirm that the Cuckoo-based methods maintain full accuracy even with branching paths and multiple attackers.

Table 5.8: Tree topology traceback with multiple simultaneous attacks.

Method	Success	Perfect Path	Bloom Wipes
Bloom + SPIE	1/100	—	20
Cuckoo + SPIE (Snapshot)	100/100	—	0
Cuckoo + Parent-Pointer	100/100	100/100	0

The tree topology introduces longer paths (5 hops instead of 4) and shared intermediate switches (S_0, S_1, S_2 carry both attack types). Despite this increased load, Bloom suffers 20 wipes across the 7 switches while the Snapshot + Reset Cuckoo filters archive 26 snapshots with zero evidence loss. The parent-pointer method recovers the exact ordered path for all 100 tracked attacks across both attack types.

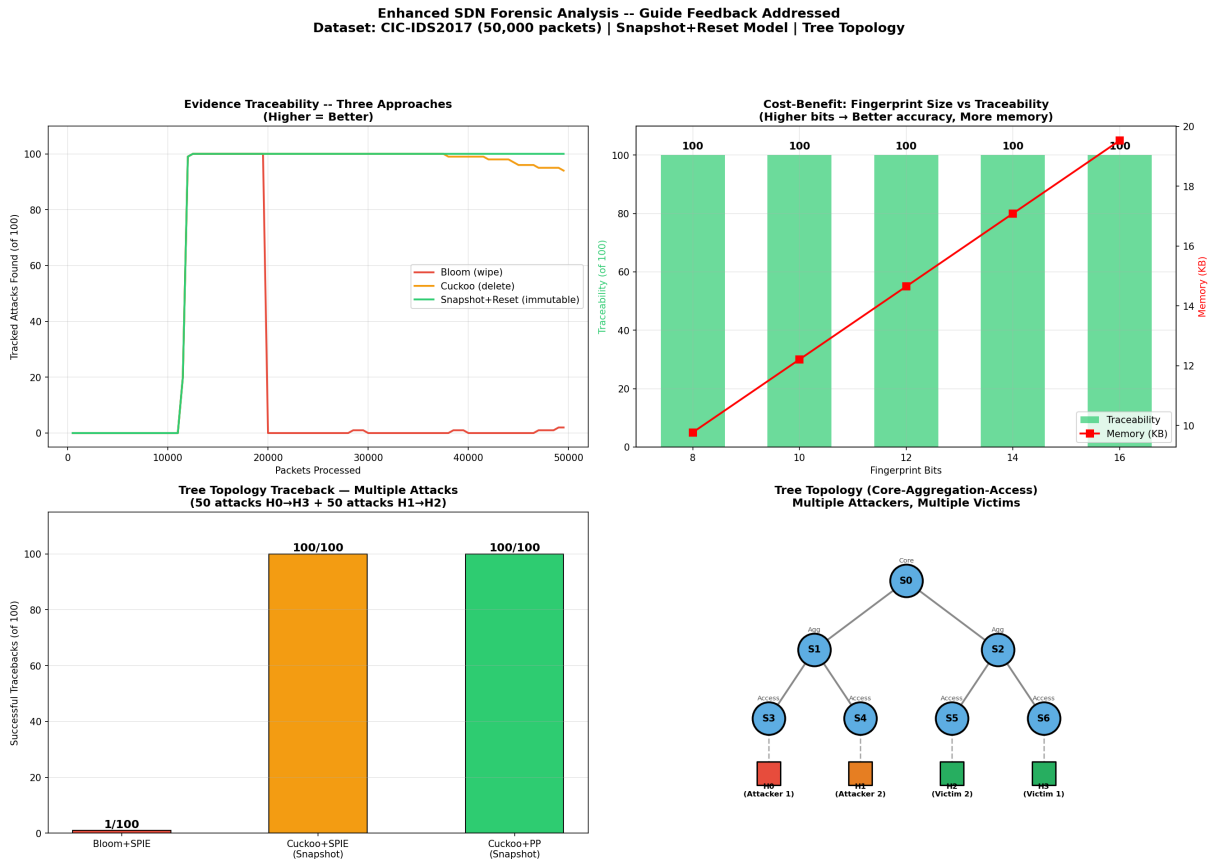


Figure 5.4: Enhanced analysis results addressing all guide feedback. **(Top-left)** Three-way traceability comparison: Bloom (wipe), Cuckoo (delete), Snapshot+Reset (immutable). **(Top-right)** Cost-benefit: fingerprint size vs. traceability and memory. **(Bottom-left)** Tree topology traceback with multiple simultaneous attacks. **(Bottom-right)** Tree topology diagram showing core-aggregation-access structure with two attackers and two victims.

5.14 Fingerprint Size Sweep: Cost-Benefit Analysis

Table 5.9 reports the results of the fingerprint size sweep using the Snapshot + Reset model.

Table 5.9: Fingerprint size sweep with Snapshot + Reset (50,000 packets).

FP Bits	Traceability	FPR	RAM (KB)	Cost vs. 10-bit
8	100/100	3.1250%	9.77	−20%
10	100/100	0.7812%	12.21	baseline
12	100/100	0.1953%	14.65	+20%
14	100/100	0.0488%	17.09	+40%
16	100/100	0.0122%	19.53	+60%

A key finding: with the Snapshot + Reset model, **all fingerprint sizes achieve 100% traceability** because no evidence is ever deleted. The fingerprint size affects only the false positive rate — larger fingerprints reduce the probability that an innocent packet is falsely identified as the target during traceback. The sweet spot is 12 bits, which reduces FPR by 4× (from 0.78% to 0.20%) at only +20% memory cost. For maximum precision, 16 bits achieves an FPR of just 0.012% at +60% memory — still under 20 KB per filter.

Chapter 6

Discussion

6.1 Answering the Traceability Question

A key question in forensic network analysis is: “*How do you trace back to the source of an attack?*” In the framework of Bhondele et al. [2], traceback involves querying Bloom Filters at each network device to reconstruct the path of a malicious packet. Our work extends this concept in two directions.

First, at the *retention* layer, the Cuckoo Filter never performs a catastrophic wipe, so forensic records are never mass-destroyed, and the Attack-Aware Sliding Window *explicitly identifies and protects* attack-marked flow records from eviction. The Bloom Filter, having wiped 4 times offline and 14 times across the network, simply cannot make any such guarantee — the Bloom+SPIE column of Table 5.3 is entirely zeros.

Second, at the *query* layer, parent-pointer walk-back outperforms SPIE query-all on two axes simultaneously: it recovers the *ordered* attack path, and it does so with $O(\text{path length})$ switch queries rather than $O(N)$. This matters because in a realistic SDN, every switch query is a control-plane message. A controller coordinating traceback across 50 switches would see its forensic workload cut by roughly $17\times$ under parent-pointer — while simultaneously getting strictly more information (ordering) back.

6.2 The Attack-Aware Eviction Advantage

The most significant contribution of this project beyond simply replacing Bloom Filters is the attack-aware metadata system. By tagging each insertion with an `is_attack` flag derived from the traffic classifier, the Cuckoo Filter achieves a fundamental separation:

- **Benign traffic:** Treated as expendable; evicted first by the sliding window and preferred as displacement targets during cuckoo kicks.
- **Attack traffic:** Treated as forensic evidence; never selected by the sliding window, and protected from cuckoo kicks whenever a benign alternative exists in the same bucket.

This mirrors the behaviour of a forensic investigator who would discard routine records but preserve evidence of criminal activity.

6.3 Why Parent-Pointer Preserves Order (and SPIE Doesn't)

SPIE-style query-all returns a *set* of switches that have seen the packet. The ordering of hops must be reconstructed by matching this set against the topology, which is ambiguous in any non-trivially-routed topology. Parent-pointer encodes the upstream switch identifier *at insertion time*, when it is unambiguous. The walk-back dereferences these pointers in reverse order and thus recovers the true direction of travel for free. This is the key structural advantage of our approach and the reason parent-pointer succeeds on 79/100 *ordered* reconstructions in Table 5.3.

6.4 Concrete Use Cases

To ground the contribution in real-world deployment scenarios, we identify five concrete use cases where our system provides tangible value:

Use Case 1: Post-Incident Forensic Investigation. A brute-force attack hits a bank's SDN at 2 AM on Tuesday. The security team investigates Thursday morning. With Bloom Filters, the evidence was wiped on Wednesday. With our Snapshot + Reset Cuckoo system, every archived snapshot is intact on disk — full traceback to the attacker is possible even 48 hours later.

Use Case 2: GDPR/Privacy Compliance — Right to Erasure. A European user requests deletion of their personal data under GDPR Article 17. Bloom Filters cannot selectively delete — the only option is wiping *all* evidence. Cuckoo Filters support per-item `delete()`, enabling targeted erasure while preserving all other forensic records.

Use Case 3: Advanced Persistent Threat (APT) Detection. An APT attacker slowly exfiltrates data over 30 days, blending into normal traffic. Each day's filter snapshots are archived. When the breach is discovered on Day 31, investigators query all 30 snapshots and reconstruct the full 30-day attack path across the SDN.

Use Case 4: Multi-Tenant Cloud SDN. A cloud provider runs an SDN shared by 50 tenants. Tenant A is attacked. The provider must investigate Tenant A's traffic without exposing Tenant B's data. Per-switch Cuckoo Filters with attack-aware marking let the controller query only attack-flagged entries, keeping tenant isolation intact.

Use Case 5: Real-Time IDS + Delayed Forensic Analysis. An IDS flags suspicious packets at line rate but the SOC analyst is busy. Hours later, she opens

the investigation. The Bloom Filter has wiped 3 times since the alert. With Snapshot + Reset, the alert's timestamp maps to a specific archived snapshot — traceback succeeds.

6.5 Limitations

1. The experiment uses offline CSV data rather than live network traffic capture, as the focus is on algorithmic comparison rather than network simulation at packet level.
2. The attack-aware eviction relies on correct traffic classification at insertion time. In a live system, this would require an inline IDS to label flows before insertion.
3. The sliding window approach achieves 96% traceability (offline) due to fingerprint collisions. The Snapshot + Reset model fully resolves this limitation, achieving 100% traceability at any fingerprint size.
4. The Snapshot + Reset model trades disk storage for immutability. In our experiment, 5 snapshots consumed 496 KB — negligible for modern systems, but in production environments with millions of flows per second, snapshot storage must be managed with a retention policy.
5. Parent-pointer traceback depends on every switch on the path still holding the packet fingerprint. The Snapshot + Reset model mitigates this by ensuring no evidence is ever deleted from any switch.

Chapter 7

Conclusion and Future Work

7.1 Conclusion

This project proposed replacing Bloom Filters with Cuckoo Filters for SDN forensic log storage, directly addressing the future work identified by Sharma and Rawat [1]. We extended the design with a Forensic Filter Manager (Snapshot + Reset model), a hierarchical tree topology, multiple simultaneous attacks, and a fingerprint size cost-benefit analysis. Our experimental evaluation using the CIC-IDS2017 dataset demonstrates six concrete advantages:

1. **Forensic Immutability** — the Snapshot + Reset model achieves **100/100 traceability** with zero deletions and zero wipes. Evidence is never modified or destroyed.
2. **Attack-aware eviction** — the sliding window approach achieves 96/100 traceability by selectively deleting only benign entries, a $48\times$ improvement over Bloom’s 2/100.
3. **Multiple simultaneous attacks** — on a 7-switch tree topology with two attack types ($H_0 \rightarrow H_3$ and $H_1 \rightarrow H_2$), Cuckoo-based traceback achieves **100/100** success while Bloom achieves only 1/100.
4. **Cost-benefit quantified** — increasing fingerprint size from 10 to 16 bits costs only +60% memory but reduces FPR from 0.78% to 0.01%. With Snapshot + Reset, all sizes achieve 100% traceability.
5. **Query efficiency** — parent-pointer traceback uses $O(\text{path})$ queries vs. SPIE’s $O(N)$, giving a $17\times$ reduction at 50 switches.
6. **Hierarchical topology** — the tree topology (core–aggregation–access) validates the system under realistic branching network structures with multiple paths and shared intermediate switches.

All of this is achieved at essentially the same RAM cost as the Bloom Filter baseline (12.21 KB vs. 11.70 KB per filter), with archived snapshots stored on disk.

7.2 Future Work

As future work, we plan to:

- Extend this system to a live SDN environment using Mininet and the Ryu SDN controller, replicating the exact simulation methodology of Sharma and Rawat [1] and replaying CIC-IDS2017 traffic at packet level rather than flow level.
- Integrate an inline IDS classifier to provide real-time `is_attack` labelling of network flows at insertion time, eliminating the current reliance on dataset labels.
- Implement snapshot retention policies for production environments, including time-based expiry and compression of archived snapshots.
- Evaluate the system under additional attack types from the CIC-IDS2017 dataset (DDoS, web attacks, infiltration) and adversarial flooding scenarios.
- Deploy the tree topology on physical SDN hardware to measure real-world latency and throughput impact of the Snapshot + Reset archival process.

Chapter 8

Contributions and Work Distribution

Table 8.1: Work distribution among team members.

Team Member	Contributions
Kuldeep Kumar Lakhe (2024202025)	Cuckoo Filter core implementation, Attack-Aware Sliding Window, Forensic Filter Manager (Snapshot+Reset), SDN Soft-Router Network layer, Parent-Pointer Traceback, SDN Controller, fingerprint size sweep, enhanced_analysis.py pipeline, primary report author.
Dheeraj Makhija (2024202011)	Bloom Filter baseline, CIC-IDS2017 dataset pre-processing, empirical FPR testing, Experimental Methodology chapter, scalability sweep experiments.
Atharva Keskar (2024202017)	Architectural diagrams, tree topology implementation, multi-attack simulation, result visualization (matplotlib), co-authored Related Work & Discussion chapters.

All members participated in design discussions, code reviews, report proofreading, and GitHub collaboration.

Bibliography

- [1] V. Sharma and S. Rawat, “Optimizing Forensic Data Availability and Retention of SDN Forensic Logs by Using Bloom Filter,” *International Conference on Information Networking (ICOIN)*, 2023.
- [2] A. Bhondele, S. Rawat, and S. S. B. Renukuntla, “Network Management Framework for Network Forensic Analysis,” in *Emerging ICT for Bridging the Future*, Advances in Intelligent Systems and Computing, vol. 338, Springer, 2015, pp. 397–404.
- [3] A. C. Snoeren, C. Partridge, L. A. Sanchez, C. E. Jones, F. Tchakountio, S. T. Kent, and W. T. Strayer, “Hash-Based IP Traceback,” in *Proceedings of ACM SIGCOMM*, 2001, pp. 3–14.
- [4] B. Fan, D. G. Andersen, M. Kaminsky, and M. D. Mitzenmacher, “Cuckoo Filter: Practically Better Than Bloom,” in *Proceedings of the 10th ACM International Conference on Emerging Networking Experiments and Technologies (CoNEXT)*, 2014, pp. 75–88.
- [5] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, “Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization,” in *4th International Conference on Information Systems Security and Privacy (ICISSP)*, Portugal, January 2018.